

ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 23/06

- Arreglos Unidimensionales: Búsquedas.
- Arreglos Unidimensionales: Mezcla.
- Arreglos Unidimensionales: Ordenamiento.

2

Algoritmos y Estructuras de Datos – AEDD – Calendario

Instancias de Evaluación	Parte	Fecha
Parcial 2	Parte Teórica Escrita	03/08/2026
	Parte Práctica Escrita	03/08/2026
	Parte Sitio Juez	03/08/2026
Parcial 2 - Recuperatorio	Parte Práctica Escrita	21/08/2026
	Parte Sitio Juez	17/08/2026
Parcial 3	<i>En el mes de septiembre (final) (fecha a confirmar)</i>	

3

Algoritmos y Estructuras de Datos – Temas del parcial 2

- ✓Funciones.
- ✓Funciones recursivas.
- ✓Arreglos unidimensionales.
- ✓Búsqueda y ordenamiento.

4

Algoritmos y Estructuras de Datos – AEDD – Asistencia 23/06

Contraseña del estudiante
2hxjl6

5

Operaciones con arreglos

- **S**earch – Búsqueda: Secuencial y Binaria (para arreglos ordenados).
- **M**erge – Mezcla – Intercalado: de dos arreglos ordenados.
- **S**ort – Ordenamiento: Algoritmos básicos y avanzados.

6

Search - Buscar un elemento en un arreglo

Buscar un elemento en un arreglo: Dados un arreglo $A[N]$ y un elemento X , determinar si existe algún i , tal que $0 \leq i < N$ y $A[i]=X$

Cuando A ni esta ordenado, se realiza una **Búsqueda Secuencial**.

- Mejor caso: 0 pasadas del ciclo while
- Peor caso: N pasadas del ciclo while

```
//Esta funcion devuelve la ubicacion de clave en la lista
//Se devuelve -1 si no se encuentra el valor
int busquedaLineal(const int vec[], int TL, int clave)
{
    int esta=0;
    int i=0;
    int pos=-1;

    while (!esta && i<TL){
        if (vec[i]==clave){
            esta=1;
            pos=i;
        }
        else
            i++;
    }
    return pos;
}
```

7

Search - Buscar un elemento en un arreglo ordenado

Buscar un elemento en un arreglo: Dados un arreglo $A[N]$ y un elemento X , determinar si existe algún i , tal que $0 \leq i < N$ y $A[i]=X$

Si A está ordenado: **Búsqueda Binaria**

- Mejor caso: 0 pasadas del ciclo while
- Peor caso: $\log_2 N$ pasadas del ciclo while

```
int busquedaBinaria(const int vec [], const int tl, const int valor){
    // busqueda binaria en vector ordenado
    int central, bajo, alto;
    bajo=0;
    alto=tl-1;
    //pos=-1;

    while (bajo <= alto){
        central = (bajo + alto) / 2;

        if (vec[central] == valor)
            return central;
        else
            if (vec[central] > valor)
                alto = central-1;
            else
                bajo = central+1;
    }
    return -1;
}
```

8

Merge – Mezclar dos arreglos ordenados

```
int main(){
    int vector1[TF], vector2[TF], vector3[TF*2];
    int cant1, cant2, cant3;

    cout << "Tamaño Logico del vector 1? ";
    cin >> cant1;
    cargarvector(vector1, cant1); //ordenado
    mostrarvector(vector1, cant1);

    cout << "Tamaño Logico del vector 2? ";
    cin >> cant2;
    cargarvector(vector2, cant2); //ordenado
    mostrarvector(vector2, cant2);

    intercalar(vector1, cant1, vector2, cant2, vector3, cant3);
    mostrarvector(vector3, cant3);
}
```

9

Merge – Mezclar dos arreglos ordenados

```
void intercalar(int vector1[], const int cant1, int vector2[], const int cant2,
               int vector3[], int& cant3){
    int i=0, j=0, k=0;

    while (i<cant1 && j<cant2){
        if (vector1[i]<vector2[j]){
            vector3[k]=vector1[i];
            i++; }
        else {
            vector3[k]=vector2[j];
            j++; }
        k++;
    }
    while (i<cant1){
        vector3[k]=vector1[i];
        i++; k++; }
    while (j<cant2){
        vector3[k]=vector2[j];
        j++; k++; }
}
```

10

Contraseña del estudiante

2hxjl6

11

Sort - Ordenar – Definición

Ordenar: significa reagrupar o reorganizar un conjunto de datos en algún determinado orden con respecto a uno de los campos del conjunto. El campo por el cual se ordena un conjunto de datos se denomina *clave*.

Formalmente, se define la ordenación de la siguiente manera:

Sea A una lista de N elementos: A1, A2, A3 An

Ordenar significa permutar estos elementos de tal forma que los mismos queden de acuerdo con un orden preestablecido.

Ascendente: $A1 \leq A2 \leq A3 \dots \leq An$

Descendente: $A1 \geq A2 \geq A3 \dots \geq An$

12

Conceptos de Orden

Ordenación interna: Los datos se encuentran almacenados en memoria (ya sean arrays, listas, etc), y son de acceso aleatorio o directo (se puede acceder a un determinado campo sin pasar por los anteriores).

Ordenación externa: Los datos están en un dispositivo de almacenamiento externo (archivos), y es probable que no tengan acceso directo. Su ordenación es más lenta que la interna.

13

Arreglos Lineales: Ordenación Interna

Métodos simples

- Selección directa
- Inserción directa
- Burbuja
- Burbuja mejorado con señal

Métodos avanzados

- Merge sort

14

Método de selección directa

Descripción:

Se debe encontrar el elemento más pequeño del arreglo e intercambiarlo por el elemento de la primera posición, luego encontrar el segundo elemento más pequeño e intercambiarlo con el elemento de la segunda posición y continuar de esta manera hasta que se haya ordenado el arreglo entero.

Los pasos del algoritmo son:

1. Seleccionar el menor elemento del arreglo. Intercambiarlo con el primer elemento $A[0]$.
2. Considerar el resto de las posiciones del arreglo ($A[1], A[2], \dots$) Seleccionar el menor elemento e intercambiarlo con el elemento $A[1]$.
3. Repetir los pasos anteriores con los elementos restantes del arreglo, seleccionando el elemento más pequeño e intercambiándolo adecuadamente.

15

Método de selección directa: Ejemplo

Ejemplo: Consideremos un arreglo B con 8 valores enteros:

25	3	12	19	2	1	9	6
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 1: Seleccionar 1 e intercambiarlo por B[0]

1	3	12	19	2	25	9	6
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 2: Seleccionar 2 e intercambiarlo por B[1]

1	2	12	19	3	25	9	6
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 3: Seleccionar 3 e intercambiarlo por B[2]

1	2	3	19	12	25	9	6
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 4: Seleccionar 6 e intercambiarlo por B[3]

1	2	3	6	12	25	9	19
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 5: Seleccionar 9 e intercambiarlo por B[4]

1	2	3	6	9	25	12	19
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 6: Seleccionar 12 e intercambiarlo por B[5]

1	2	3	6	9	12	25	19
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

Pasada 7: Seleccionar 19 e intercambiarlo por B[6]

1	2	3	6	9	12	19	25
B[0]	B[1]	B[2]	B[3]	B[4]	B[5]	B[6]	B[7]

16

Método de selección directa: Código

```

void seleccion(int vec[], const int tl){
    int i,j,min;

    for (i=0; i<tl; i++) {
        min=i;
        for (j=i+1; j<tl; j++)
            if (vec[j] < vec[min]) min=j;
        intercambio(vec[i],vec[min]);
    }
}

void intercambio( int& a, int& b ){
    int aux = a;
    a = b;
    b = aux;
}

```

17

Comportamiento del Método de selección directa

(Libro: "Programación en C++ para ingenieros" - Fatos Xhafa y otros)

Criterios	Estable	Natural	Orden definitivo	Eficiencia
	Valores iguales en el arreglo guardan su orden relativo	Tiene en cuenta la posible ordenación del arreglo aunque sea parcial y por lo tanto no hace intercambios	En cada iteración se pone un elemento en su sitio definitivo	
Selección Directa	No	No	Si	Cuadrática

18

Método de inserción directa

Descripción:

En este algoritmo consideramos un elemento a la vez y lo movemos hasta el lugar correcto, entre aquellos que ya han sido considerados (manteniéndolos ordenados).

El elemento se inserta moviendo los elementos superiores a él una posición a la derecha y ocupando la posición vacante.

Código:

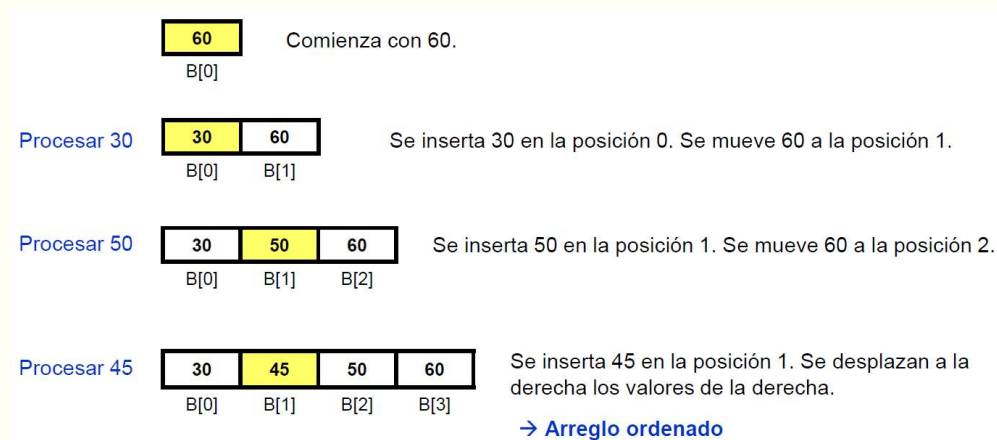
```
void insercion(int vec[], const int tl){
    int i,j;

    for (i=1; i<tl; i++) {
        j=i;
        while ((j>0) && (vec[j] < vec[j-1])) {
            intercambio(vec[j],vec[j-1]);
            j = j-1;
        }
    }
}
```

19

Ordenamiento: Método de inserción directa

Ejemplo: Consideremos un arreglo B con 4 valores enteros: 60, 30, 50, 45



20

Comportamiento del Método de inserción directa

(Libro: "Programación en C++ para ingenieros" - Fatos Xhafa y otros)

Criterios	Estable	Natural	Orden definitivo	Eficiencia
	Valores iguales en el arreglo guardan su orden relativo	Tiene en cuenta la posible ordenación del arreglo aunque sea parcial y por lo tanto no hace intercambios	En cada iteración se pone un elemento en su sitio definitivo	
Inserción Directa	Si	Si	No	Cuadrática

21

Algoritmos y Estructuras de Datos – AEDD – Asistencia 23/06

Contraseña del estudiante
2hxjl6

22

Método de la burbuja

Descripción:

- Lleva los elementos más pequeños hacia la parte izquierda del arreglo.
- Realiza repetidamente el intercambio de pares de elementos adyacentes hasta que estén todos ordenados.
- Se hacen varias pasadas a través del arreglo.
- En cada pasada se comparan pares de elementos. Si están en orden creciente (o los valores son idénticos) se los deja como está, sino se los intercambia.

23

Método de la burbuja

Ejemplo: Consideremos un arreglo B con 5 valores enteros: 60, 30, 45, 75, 15.

Pasada 1	60	30	45	75	15	Se intercambia 60 y 30.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	60	45	75	15	Se intercambia 60 y 45.
	B[0]	B[1]	B[2]	B[3]	B[4]	
Pasada 2	30	45	60	75	15	60 y 75 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	45	60	75	15	Se intercambia 75 y 15.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	45	60	15	75	30 y 45 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	45	60	15	75	45 y 60 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	45	60	15	75	Se intercambia 60 y 15.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	45	15	60	75	60 y 75 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	

24

Método de la burbuja: continuación

Pasada 3	30	45	15	60	75	30 y 45 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	45	15	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	15	45	60	75	Se intercambia 45 y 15.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	15	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	15	45	60	75	45 y 60 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	15	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	15	45	60	75	60 y 75 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	30	15	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
Pasada 4	30	15	45	60	75	Se intercambia 30 y 15.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	30 y 45 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	45 y 60 están ordenados.
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	60 y 75 están ordenados → Arreglo ordenado
	B[0]	B[1]	B[2]	B[3]	B[4]	
	15	30	45	60	75	
	B[0]	B[1]	B[2]	B[3]	B[4]	

25

Método de la burbuja

Codificación:

```

void burbuja(int vec[], const int tl){
    int pasada, k;

    for ( pasada = 1; pasada < tl; pasada++ )
        for ( k = 0; k < tl - 1; k++ )
            if ( vec[ k ] > vec[ k + 1 ] )
                intercambio( vec[ k ], vec[ k + 1 ] );
}

```

26

Comportamiento del Método de la burbuja

(Libro: "Programación en C++ para ingenieros" - Fatos Xhafa y otros)

Criterios	Estable	Natural	Orden definitivo	Eficiencia
	Valores iguales en el arreglo guardan su orden relativo	Tiene en cuenta la posible ordenación del arreglo aunque sea parcial y por lo tanto no hace intercambios	En cada iteración se pone un elemento en su sitio definitivo	
Burbuja	Si	No	No para los K primeros Sí para los K últimos	Cuadrática

27

Método de la burbuja mejorado (versión 2)

Descripción:

- Es una versión mejorada del algoritmo anterior, donde se realiza una comparación menos por cada pasada (aprovechando el hecho de que los elementos en el extremo derecho van quedando ordenados y no tiene sentido volver a compararlos).

Codificación:

```
void burbujamejorado(int vec[], const int tl){
    int pasada, k;

    for ( pasada = 1; pasada < tl; pasada++ )
        for ( k = 0; k < tl - pasada; k++ )
            if ( vec[ k ] > vec[ k + 1 ] )
                intercambio( vec[ k ], vec[ k + 1 ] );
}
```

28

Método de la burbuja mejorado con centinela

El algoritmo finaliza cuando no se produce intercambio alguno entre los elementos del arreglo, pues esto indica que ya quedó ordenado.

Descripción:

- Este algoritmo utiliza una marca o señal para indicar que no se ha producido ningún intercambio en una pasada. Al final de cada pasada se evalúa si no se han hecho intercambios y en ese caso se termina el ciclo externo.

29

Método de la burbuja mejorado con centinela

El algoritmo finaliza cuando no se produce intercambio alguno entre los elementos del arreglo, pues esto indica que ya quedó ordenado.

Codificación:

```
void burbujacentinela(int vec[], const int tl){
    int pasada, k, intercambios=1;
    pasada=1;
    while ( pasada < tl && intercambios ){
        intercambios=0;
        for ( k = 0; k < tl - pasada; k++ )
            if ( vec[ k ] > vec[ k + 1 ] ){
                intercambio( vec[ k ], vec[ k + 1 ] );
                intercambios=1;
            }
        pasada = pasada + 1;
    }
}
```

30

Análisis de rendimiento de selección directa

Este es uno de los algoritmos más simples de ordenamiento.

Cada elemento se mueve como máximo una vez. El número de intercambios realizados es menor que en el burbuja.

Este algoritmo realiza muchas menos operaciones intercambio() que el de la burbuja ($n-1$ en total).

Una desventaja respecto al burbuja con centinela es que no mejora su rendimiento cuando los datos ya están ordenados o parcialmente ordenados.

En el caso del burbuja con centinela se requiere una única pasada para detectar que el vector ya está ordenado y finalizar. En la ordenación por selección se realiza el mismo número de pasadas independientemente de si los datos están ordenados o no.

Este método es recomendable para un número pequeño de elementos.

31

Análisis de rendimiento de inserción directa

En el mejor de los casos, el arreglo estará inicialmente ordenado, entonces el bucle interno (while) sólo ejecuta el paso de comparación, que siempre será falso. Esto implica que en el mejor de los casos el número de comparaciones será $n-1$.

En el peor de los casos, el arreglo está inversamente ordenado y el número de comparaciones a realizar será el máximo.

En el caso promedio, los elementos aparecen en el arreglo en forma aleatoria, y puede ser calculado mediante la suma del mejor y peor caso dividido entre 2.

A pesar de ser un método ineficiente y recomendable solo para un número pequeño de elementos, es intuitivo y de muy fácil implementación.

32

Análisis de rendimiento de burbuja con centinela

- Si el arreglo está completamente desordenado (peor caso), el ciclo while se comporta como el ciclo for de las versiones anteriores, y la cantidad de comparaciones e intercambios realizados es similar.
- Si el arreglo queda completamente ordenado en alguna pasada intermedia, nos ahorramos todas las comparaciones de las pasadas restantes.
- Si el arreglo está inicialmente ordenado, sólo se ejecuta una pasada sobre el mismo (realizando $n-1$ comparaciones) y ningún intercambio.

33

Algoritmos y Estructuras de Datos – AEDD – Asistencia 23/06

Contraseña del estudiante
2hxjl6

34

Método Merge-Sort

Descripción:

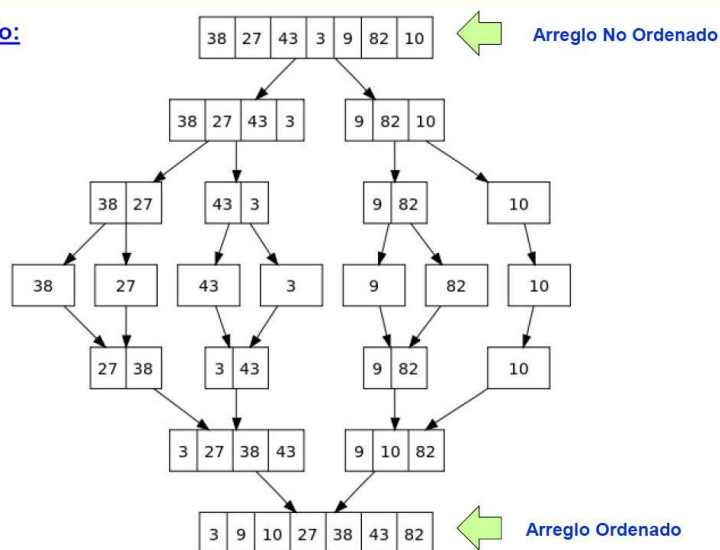
En este método se unen dos estructuras ordenadas para formar una sola, ordenada correctamente.

- Consiste en dividir en dos partes iguales el vector a ordenar, ordenar por separado cada una de las partes, y luego mezclar ambas partes, manteniendo el orden, en un solo vector ordenado.
- Tiene la ventaja de tener complejidad logarítmica: $n \log(n)$.
- Su desventaja radica en que se requiere de un espacio extra para el procedimiento.

35

Método Merge-Sort

Ejemplo:



36

Método Merge-Sort

```
#include <iostream>
using namespace std;

void mergesort(int v[], int inicio , int final);
void merge(int v[], int inicio1, int final1, int inicio2, int final2);

int main (void){
    int c [8];
    int i;

    cout << endl << "Ingrese un dato por renglon:" << endl;
    for (i=0; i<8; i++)
        cin >> c[i];

    mergesort(c,0,7);

    cout << endl << "Se muestran los datos ordenados:" << endl;

    for (i=0; i<8; i++)
        cout << c[i] << " ";
    return 0;
}

void mergesort(int v[], int inicio , int final) {
    if (final - inicio == 0)
        return;
    else {
        mergesort ( v, inicio , (inicio+final) / 2);
        mergesort ( v, (inicio+final) / 2 + 1, final );
        merge( v, inicio, (inicio+final) / 2, (inicio+final) / 2 + 1, final);
    }
}
```

37

Método Merge-Sort

```
void merge(int v[], int inicio1, int final1, int inicio2, int final2) {
    int i, j, k;
    int c[8];

    i = inicio1;
    j = inicio2;
    k = 0;

    while (i<=final1 && j<=final2)
        if (v[i] < v[j])
            c[k++] = v[i++];
        else
            c[k++] = v[j++];

    while (i<=final1)
        c[k++] = v[i++];

    while (j<=final2)
        c[k++] = v[j++];

    for (k=0; k<final2-inicio1+1; k++)
        v[inicio1+k] = c[k];
}
```

```
Ingrese un dato por renglon:
12
4
6
23
99
56
4
0

Se muestran los datos ordenados:
0 4 4 6 12 23 56 99

<< El programa ha finalizado: codigo
<< Presione enter para cerrar esta ve
```

38

Comportamiento del Método de la burbuja

(Libro: "Programación en C++ para ingenieros" - Fatos Xhafa y otros)

Criterios	Estable	Natural	Orden definitivo	Eficiencia
	Valores iguales en el arreglo guardan su orden relativo	Tiene en cuenta la posible ordenación del arreglo aunque sea parcial y por lo tanto no hace intercambios	En cada iteración se pone un elemento en su sitio definitivo	
Merge Sort	Si	No	No	$N \log N$

39

Problema para realizar: Carrera de Robots Clasificadores

Un equipo de investigación está diseñando robots que pueden organizar piezas mecánicas automáticamente. Para probar su eficiencia, necesitan simular el proceso en la computadora: cada robot implementa un algoritmo diferente de ordenamiento.

Se debe programar una aplicación que:

- Lea un valor N (cantidad de piezas, máximo 100).
- Lea N números enteros que representan los códigos de las piezas en el orden que fueron recolectadas.
- Implemente cuatro algoritmos de ordenamiento sobre el mismo conjunto de datos: Selección directa, Inserción directa, Burbuja, Merge-Sort. Muestre para cada algoritmo:
 - El vector ordenado.
 - La cantidad de comparaciones y/o intercambios realizados.

40

Cuadro comparativo Métodos de Ordenamiento

Método	Peor Caso	Mejor Caso	Características
Selección	$O(n^2)$	$O(n^2)$	Cuando el intercambio de datos es muy costoso (hace pocos swaps).
Inserción	$O(n^2)$	$O(n)$	Para arreglos pequeños o cuando se van recibiendo datos en tiempo real.
Burbuja	$O(n^2)$	$O(n^2)$	Educativo únicamente. Ineficiente en la práctica.
Burbuja con centinela	$O(n^2)$	$O(n)$	Cuando el arreglo ya está ordenado o casi listo.
Merge-Sort	$O(n \log n)$	$O(n \log n)$	Grandes volúmenes de datos. Rendimiento garantizado, pero usa memoria extra.

41

Problema para realizar: Carrera de Robots Clasificadores

Entradas:

Cantidad de piezas: 8

Códigos: 45 12 78 34 89 11 4 50

Salidas: Ordenando por Selección directa...

Vector ordenado: 4 11 12 34 45 50 78 89

Comparaciones: 28 Intercambios: 7

Ordenando por Inserción directa...

Vector ordenado: 4 11 12 34 45 50 78 89

Comparaciones: 20 Intercambios: 12

Ordenando por Burbuja...

Vector ordenado: 4 11 12 34 45 50 78 89

Comparaciones: 28 Intercambios: 14

42

Videos de Ordenamiento

Burbuja

- <http://www.youtube.com/watch?v=YKIDz1J3TSw>

Selección Directa

- <http://www.youtube.com/watch?v=TbwkDXtHgOU&feature=related>

Inserción Directa

- https://www.youtube.com/watch?v=Md39DDUN_EY

Inserción Directa vs Burbuja

- <https://www.youtube.com/watch?v=TZRWRjq2CAg>

Merge Sort

- <http://www.youtube.com/watch?v=L-E6KKmXR0o>
- <http://www.youtube.com/watch?v=cDNqk4tdvqQ>

43

Algoritmos y Estructuras de Datos – AEDD – Asistencia 23/06

Contraseña del estudiante
2hxjl6

44

Algoritmos y Estructuras de Datos – AEDD – Agenda 23/06

- ✓ Arreglos Unidimensionales: Búsquedas.
- ✓ Arreglos Unidimensionales: Mezcla.
- ✓ Arreglos Unidimensionales: Ordenamiento.

¡¡Objetivo cumplido!!!

45

AEDD – Otras tareas !!!!!

Para profundizar en el tema:

Libro:

Fundamentos de Programación C++ - Benjumea - Roldán

Capítulo 7 – Pág. 85

Libro:

C++ para ingeniería y ciencias - Gary J. Bronson

Capítulo 11 – Pág. 660

Libro:

Programación en C++ para ingenieros - Fatos Xhafa y otros

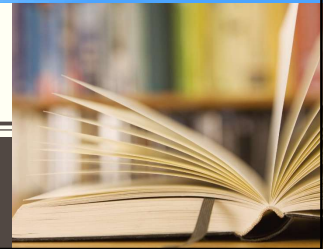
Capítulo 9 – Pág. 299

46

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



47

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Muy buena semana!



48